Computational Intelligence

In the early stages, we used basic algorithms such as Path Search, specifically the A* algorithm, to find a series of actions that solve a problem.

Later on, we transitioned to Local Search algorithms, aimed at addressing Parameter Optimization problems. The goal in this context is to maximize a given function, employing techniques like Hill Climbing, Swarm Intelligence, and Evolutionary Algorithms.

Finally, we delved into Adversarial Search, where the objective is to play against an adversary using techniques like MinMax and Reinforcement Learning (Q-function).

Monte Carlo is a method for implementing Adversarial Search that can be applied to MinMax / RL or any related approaches

Computational Intelligence

Solve problem by searching → Machine Learning
Human Learning → Trial & Error / Evolution

Path Search

in path search we return as a result a sequence of action → minimize the number of steps
In state search we return the state that maximize our result

The goal is to find a sequence of actions to solve a problem. In each step we must decide what to do next.
The idea is to model the states of a problem as nodes, so the path is from the **initial state** to the **goal states**.
Problem Space → It defines the set of all possible initial state, intermediate states, and the goal states of the problem.
Solution Space → It represents the specific sequence of actions or states that the agent needs to traverse from the initial state to the goal state to achieve a successful outcome.
State Space → A set of possible states that the environment can be in. There is the **initial space** and **one or more goal state**

The characteristics of the problems we can model:

- **sequential** → Solution as a sequence of steps
- **observable** → All relevant details are known
- **deterministic** → Effect of actions are predictable
- **static** → Time is not relevant for choices
- **discrete** → Number of possible actions is countable.

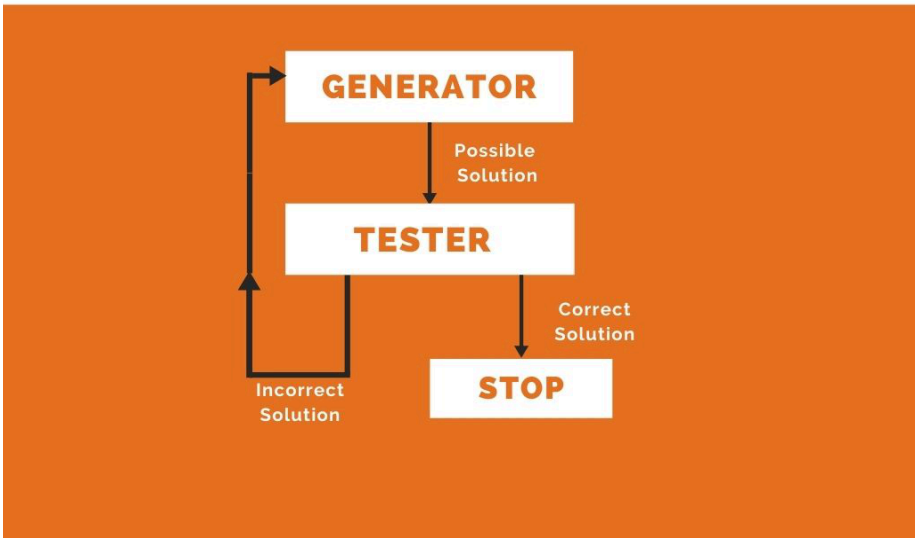
Deterministic + static implies one single player: with two or more players we can't predict the actions of our adversaries so we lose the deterministic assumption (so these algorithms can't be used).

Generate n' Test - basic approach

Generate and Test Search is a heuristic search technique based on **Depth First Search with Backtracking** which guarantees to find a solution if done systematically and there exists a solution.
In this technique, all the solutions are generated and tested for the best solution
We iterate this approach until a satisfactory solution is found.
It is a **trial and error** approach.
If done systematically (and a solution exists), this approach guarantees that a solution is found.
(heuristic search technique)
Good generators Properties:

- **Exhaustive** → all possible solutions must be generated. All the possible states must be covered
- **Not redundant** → should not generate the same solution twice. It increase exponentially the time of search exponentially
- **Informable** → Good Generators have the knowledge about the search space. This can be used to search how far the agent is from the goal, calculate the path cost and even find a way to reach the goal.

DIAGRAMMATIC REPRESENTATION



Heuristic = proceeding to a solution by trial and error or by rules that are only loosely (not strictly) defined.

Tree vs Graph search = we can choose between these two ways to represent the search problem

- Tree search
    - **Do not** remember visited nodes
    - Exponentially slower, but memory efficient

- Graph search
    - **Do** remember visited nodes.
    - Exponentially faster, but memory inefficient

State Space = a set of states that describe all the relevant details of all the possible situations.

- We need to include the **initial** state.
- **Goal Test** → function G(s) that checks if the goal has been reached in the state S
- **State vs Node** → we need to consider that in tree search a single state can be represented by multiple nodes, so there's no one-to-one relationship between node and state. A node represents a single state.

Actions → given a state *s* we defined *A(s)* as the possible actions that can be performed in *s*. An action has an Action cost function. We also define *R(s,a)* as the new state obtained by performing action *a* in state *s*.
Transition Model → The transition model defines how actions change the current state, moving from one state to another. Lookup table for checking valid transformation.
Goal state → The goal state represents what you're trying to achieve. It's where you want to end up.
Transformation → Given a complex problem, it is often convenient to transform it into something known, or at least more manageable.
Path → a sequence of actions from the initial state to the goal state. The **cost** of a path is obtained with the function C(*p*), and usually corresponds to the cost of performing the actions of the path.
Optimal solution → minimum cost path from the initial state to a goal state

Problem **solving Agents**

- **Reflex agents** → simply consider the current situation. called Greedy
- **Goal Based agents** → Consider the desirability of the future outcome of their actions. It is called "intelligent" because they maximize their performance on the long run
- **problem solving agents** → Look for actions sequence for satisfying the goal

Terminology:

- **Node** : state in the possibility infinite state space
- **Reached** : a state for which a node has been generated
- **Expand** : generating new child (successor) of the node (s)
- **Frontier** : set of reached, yet not fully expanded nodes
- **Redundant paths** : worse way to reach same state (e.g. cycle)
- **Breaching factor** : number of successor nodes considered

Generic Search → We will use a generic algorithm for searching, the differences between all the variants lies in the **priority queue function**. This is the algorithm in pseudo-code:

- While we have not reached the goal
    - get Node from priority queue
    - For all possible actions in the node
        - we perform the action and we obtain a new_state
        - if the new_state has not been already explored:
            - Add it to the frontier (list of NOT explored nodes)
        - if the cost of this path is lower:
            - update the cost of the path to this node

Over the solution space we will place a tree or graph based on which methodology we choose.

Uninformed search

Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree. In this kind of search strategy we don't have any other additional knowledge about the problem, for example there is no heuristic function. We use a trial and error approach.

Disclaimer: What makes the strategy uninformed is that we don't know the distance to the final solution. Note that the cost doesn't make the search strategy informed.

Depth First Search → the deepest node (the one with the most number of levels) in the frontier is expanded. So the priority function is directly proportional to the depth (the deepest the better) and can be modeled by using a stack.

Breadth First search → the expansions uses the less deep node, so it is essentially a level-based search. In this case, the PriorityQueue used to keep track of the node to explore is a simple list (where nodes are added using an append-only strategy).

Uniform-cost search → It is also called Dijkstra's algorithm, we expand the node with the lowest cost first. If the cost is the same for all the vertices, this is the same as breadth-first approach.

Depth-limited search → same strategy of depth-first but we add a *depth limit*. This strategy, to be a complete-search strategy, needs **iterative deepening** (after exploring everything we decrease the depth limit).

Beam Search → It is a uniform-cost search with a limited number of nodes in each level. Not a complete strategy.

Bidirectional-search → Search forward from initial states and backward from the goal state

Note: an incomplete search strategy means that it doesn't fully explore the search space (we may not consider some solutions).

Informed search

In this type of search-strategy we adopt a heuristic function so we have an approximated idea of the distance between the current node and the final solution. This doesn't mean that this initial distance we know will be the final distance.

Greedy best-first → we expand the node with the maximum expected value first.
We are considering only the heuristic function.
This strategy is not optional nor complete.

A* search → The priority uses a combination of g(n), which is the actual cost, and h(n) which is the heuristic function. In practice:

$$f(n) = g(n) + h(n)$$

A* is both a **complete and efficient strategy** (meaning that it finds the minimum cost path by expanding the minimum number of nodes) under reasonable *assumptions*.
The most important assumption is that the **heuristic must be admissible**:

- ➔ the heuristic *h(n)* must **never overestimate** the cost to reach the destination node, meaning that if the actual cost is *x* the heuristic function must always find a value that is less or equal to *x*.

The heuristic has also to be both **consistent** (aka monotonically decreasing) and **admissible**.

H it is **consistent** if for every node n and for every successor node n_s :

$$h(n) < cost(n, n\_s) + h(n\_s)$$

Note that when the heuristic is admissible it is also consistent.
Note: when there is an infinite branch factor, A* doesn't work because we are still expanding the successors of a node.

Expert systems (NON CREDO LO ABBIA VISTO IL PROF)
Ruled based system. Get knowledge from experts in the form of simple rules.

- **Forward-chain rule-based expert system** → fixed rules, no learning from experience. Both OR and AND rules.
- **Knowledge engineer** → list specific cases. Ask about things that look the same, but are handled differently

Single State Methods

Sometimes we are only interested in the solution, the path may be trivial given the solution or completely irrelevant.
Single-state methods in computational intelligence refer to techniques that operate within a single solution state to optimize or solve a given problem.
These methods don't maintain or explore multiple potential solutions simultaneously but rather **focus on refining and improving a single solution iteratively until a satisfactory solution is found.**

Single state alg. can be exhaustive and they can try all the solutions possible. They are better compared with local search which search only in a local space (not global)

Algorithm: Hill climbing - Single State Local Search - Simulated Annealing - Tabu Search - Iterated Local Search - ES

Local Search algorithm → given a task they try to find the best solution measured by a fitness function.
We can see that in this approach we don't care about a path to get to a final state, but we are interested in finding a solution that maximizes fitness.
In local search we start from an initial solution and then we explore the **neighboring state** and we update the local optimum if these solutions are better (according to the fitness function).

An example of local search algorithms are **Evolutionary Algorithm** and **Hill Climbing**

Hill climbing

Hill climbing algorithm = Given a state, **tweak** it until you find a better one according to the fitness. Quickly climb up to the first local optimum (no good).
The problem with hill climbing is that it only uses exploitation and not exploration.
Quickly climb up to the local oiptimum

$S \leftarrow$ some initial candidate solution
repeat
  $R \leftarrow$ Tweak(Copy(S))
  if Quality(R) $>$ Quality(S) **then**
    $S \leftarrow R$
until S is the ideal solution or we have run out of time
return S

Tweaks = A random change in the parameters of the sample.
Small changes favor exploitation while big ones favor exploration.

An example of Hill Climbing that tries to escape local optimum is Hill climbing with Simulated Annealing

Simulated annealing : Accept worsening changes hoping to escape local optimum. To choose the worst result we use a probability.
This probability depends on:

1. How much worse is the new solution
2. The *temperature* → the higher the temperature the higher the probability of choosing a worst result. Usually it decreases during the training of the algorithm (in the beginning we explore, then we exploit).

Tabu Search: Another improvement of Hill Climbing is **tabu search**: the idea is to create a list called *tabu_list* where we add solutions that we don't want to re-evaluate later. The idea

is to guide our algorithm in the search by not considering part of the solution space already explored and that we want to avoid (example a local optimum).

Iterated Local search : cleverer version of Hill-Climbing with Random Restart.
The process restart dorm the point selected by a random function. It works better than basic hill climbing

Exploration VS Exploitation

Exploitation → Our goal is to find the local optimum by creating new samples which are very close to the previous ones. We focus on the local optimum.
Exploration → We want to explore our global search space to understand if we can find a better solution and escape the local optimum.

Evolutionary Algorithm

Evolution is a sequence of two contributions : **variations** (mostly random) and **selection** (mostly deterministic).
In evolution, changes are not designed but **evaluated**

- is **not an Optimization process**,
- does **not have a goal**
- does **not favor strength**
- does **not favor intelligence**

However when **all variations** are accumulated in **one specific direction**, the final outcome looks like the product of an intelligent design.

Algorithms based on **biological approaches:**

- Initialization
- Breeding
- Slaughtering

These types of algorithms are based on the accumulation of tiny variations. Evolution is a sequence of **two contributions**:

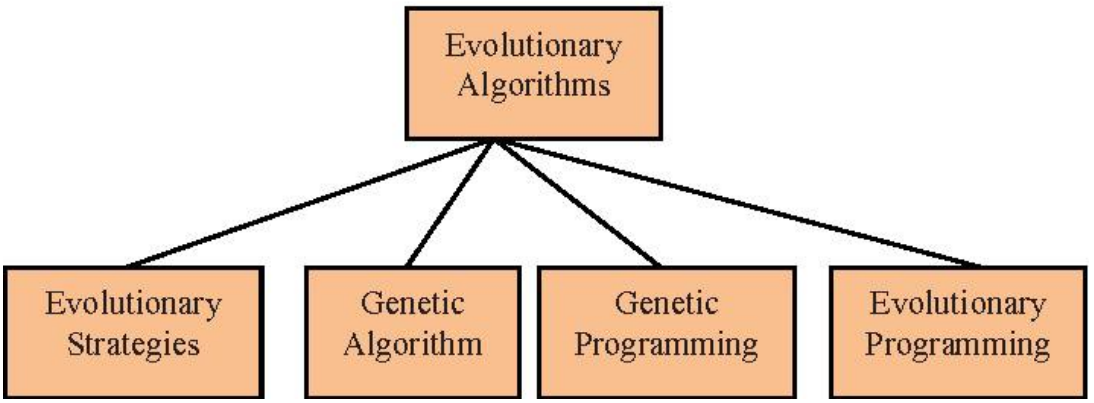
- Variations (mostly **random**)
- Selection (mostly **deterministic**)

Important: The changes are not designed (because at first we don't have an idea on how to solve the problem) but only evaluated. The evolution process is not random by itself, but to get to the solution you have to introduce some variability.

In EAs we use different terms:

- **Individual** = candidate solution, encoded in a useful way depending on the problem to solve (list of real numbers, bit string, path, etc…)
- **Population** = set of candidate solution
- **Fitness** = evaluation or ability to solve the problem
- **Generation** = sequence of steps

There are 4 types of evolutionary algorithms: Evolutionary Strategies, Genetic Algorithms, Genetic Programming and Evolutionary Programming.



Evolutionary Strategies

Evolutionary Strategy (ES) is an approach used for **local-search** problems based on EA. The idea is that each solution is a set of real numbers that is given as inputs to a function. Then, the goal is to find the individual that maximizes the function output.
This can be seen as a **parameter-optimization** approach. The individual encoding in ES is always done as a list of real numbers.

ES variants

There are three different possible variants of Evolutionary Strategies, written in the following way:

(... *[symbol]* …) = the first part represents the **population size**, while the second the **offspring size**

Depending on the *[symbol]*:

- If *[symbol]* is a +, then there is a competition between offsprings and the parents
- If *[symbol]* is a , then you replace directly who generated the offsprings

For example:

- **(1+1)**
  - *First-improvement* Hill-Climber
  - You do a competition between one parent and one offspring
  - The tweak is based on a gaussian mutation on each element
  - The values created by the gaussian depend on the *mutation step*
- **(1+λ)**
  - A *steepest-step* (looks at different possible directions) Hill-Climber
  - Given an individual, we make a single parent compete with multiple individuals
- **(1,λ)**
  - A *steepest-step* Hill-Climber
  - Given an individual, we replace the parent with one offspring chosen between **λ** even if all the new individuals perform worse than the parent

    - The idea is to escape a local optimum
- **(μ,λ)**
    - A population of μ individuals where in each generation we select the best μ individuals out of λ offsprings
    - The offspring are generated by mutating randomly the μ parents
    - Self-adaptive strategy: parameter value + mutation step

Strong causality = There is a good correlation between the parameters and the final value of the fitness. A small change in the parameters implies a small change in the fitness function. If we keep going in the same direction we can expect the fitness values to grow.
Weak causality = a small change in the parameters can end up with higher changes in the fitness. In this case, if we apply a small change the fitness value can both increase or decrease, so we have an unpredictable outcome.

Dynamic Strategy: The *mutation step* (SIGMA) is not fixed, but we can adapt it using a dynamic strategy (also called **adaptation**). The main way to adapt the mutation step is monitoring the number of successes over a given amount of steps (**ERA**) and changing it to create a *balance between exploration and exploitation*.

(rivedere meglio)
An alternative to adaptation is self-adaptation. The idea of **self-adaptation** is that now the mutation-step is included as a parameter for each individual.
At each simulation step, we tweak the mutation-step and then, based on it, we create the new offspring.
So, a new way to introduce the recombination factor in the representation is: **(μ/ρ,λ)**

Genetic Algorithm - Fraser → Holland

The second type of EA is Genetic Algorithm. This approach is more biologically-based and it is used to solve a wider range of problems compared to ES. because we are not bound to *parameter-optimization problems*.

The **individual encoding in GA depends on the problem we are solving**. It can be a list of real numbers, bit strings, permutations and much more. An individual it is unique represented

GA terminology

Genome = encoding of a possible solution
Gene = represents an element of the genome
Locus = position of a specific gene inside its genome
Allele = domain of possible values for a gene (*for example, if the gene in the second position has a true/false value, then the domain of possible values is true/false*)

The main genetic operations that we can perform on offspring are **recombination** and **mutation**. After those two operations, the biological-idea used here is that the offspring must inherit the main characteristics of its parents.

Genetic Operations : Recombination & mutation

The offspring must inherit the main characteristics of the parents

Recombination

The idea of recombination is that, instead of mutation where the generation of the offspring uses a single parent, we use two or more parents. The *recombination factor (ρ)* is the number of parents.

Crossover

One way to perform recombination is crossover. **Crossover** is a strategy that allows us to build an offspring genome by choosing each gene from one of the parents. We can choose how we pick the offspring's gene using different techniques (single gene, one-cut, etc…)

Survival Selection

Survival selection is fully **deterministic**, so it is still based on the **fitness function**.

Parent Selection

For **parent-selection**, we don't only choose the best parents because it is risky. The problem with that approach is that we could choose two very similar parents. Two similar parents would lead to an offspring that is almost-exact copy of its parents. The idea we will use for parent-selection is a tournament-based selection, which is the one that performs better**.**

Two strategies for parent selection:

- **Roulette wheel** : strategy for parent-selection. In this case there is a probability involved in the selection of parents. In general, this probability is computed by taking into consideration the fitness values. Probability of being chosen increase with high fitness values
- **Tournament selection** involves running several "tournaments" or competitions among a pool of individuals chosen at random from the population. The winner of each tournament (so, the one with the best fitness) is selected for crossover.
    - In tournament selection we can define a **selective pressure** that depends on the number of individuals participating at the tournament. The larger the population is, there is a higher probability that a stronger individual is also in that tournament so the selective pressure increases. By using selective pressure we are implicitly trying to balance between exploration and exploitation.

Alternative types of GA

- **Strategy 1 - Recombination + mutation**
    - Select the fittest individuals
    - Apply mutation depending on a *mutation rate* (i.e. how many people can mutate)
    - Do recombination through crossover to generate new offsprings
- **Strategy 2 - Recombination vs mutation**
    - Select the fittest individuals

    - Choose whether to apply recombination or mutation based on probability until you have generated an offspring

Once you create an offspring, we have two options:

- **Steady state** = there is a **competition** between the old population and the offsprings. Only the fittest individuals of the two sets (old population and new offsprings) will survive.
- **Generational** = the new offsprings will entirely replace the old population.
- **Generational + elitism** = take only the best individuals that are generated and I copy them in the next generation.

Other considerations

The first thing we should consider is: what if the individual is an **invalid solution**? There can be three ways to solve this issue: **discard** the individual, **repair** the individual or **accept** it but add a penalty.
Another thing to consider is **tuning,** so how do we set *population_size*, *offspring_size, tournament_size*? → starting the algorithm a lot of times for finding the Hyperparameters
Also, how do individuals pair with each other? Are all individuals potential partners for each other (**panmictic**) or is there a predefined rule (**lattice**)?

Stopping condition

In order to stop the simulation, we have to choose the criteria on which we decide to stop.
We can stop the simulation when:

- we have found the best solution (hard)
- based on the total number of evaluations
- total number of steps, useful in parallel environments
- wall-clock time
- **steady-state,** when the chance of improvement is low we stop.

TSP → Travelling Salesman Problem
A vendor must visit all the cities in the shortest time possible .
Name of the locations and distances are known

- The shortest possible route that visits each city exactly once and returns to the origin city
- Shortest Hamiltonian Cycle

More individual can play different route

Holland's schema theory

The idea of this theory is to study genetic algorithms by picking a set of common individuals with common traits. These common traits are called **schema** and they are represented as a subset within the solution space.
Some short and high-performance schemata are called **building blocks**. Those building blocks are used in GAs to seek near optimal performance. Combining building blocks together can lead to optimal solutions.

Genetic Programming

Genetic Programming was born as a general framework for Evolutionary Computation.
The idea is to build a formula to solve a problem modeled like this:

IF formula THEN good ELSE bad

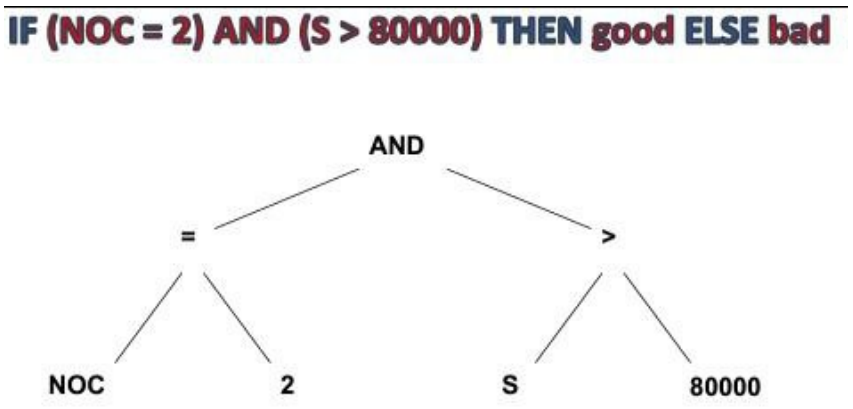
We want to build *formula* by applying an EA approach.
Genetic Programming can be used to represent computer programs as a tree

GP terminology

We need a structured way to represent a formula.
Genotype : structured **representation of the formula** (parse tree).
Sometimes, when evaluating the individual, we can't directly use the genotype (meaning we can't directly use its representation) so we need an intermediate representation called **phenotype.**

Phenotype → **formulas** (the actual formal expression generated from the genotype)
Fitness function : % of success

The formula is represented as a **parse-tree:**



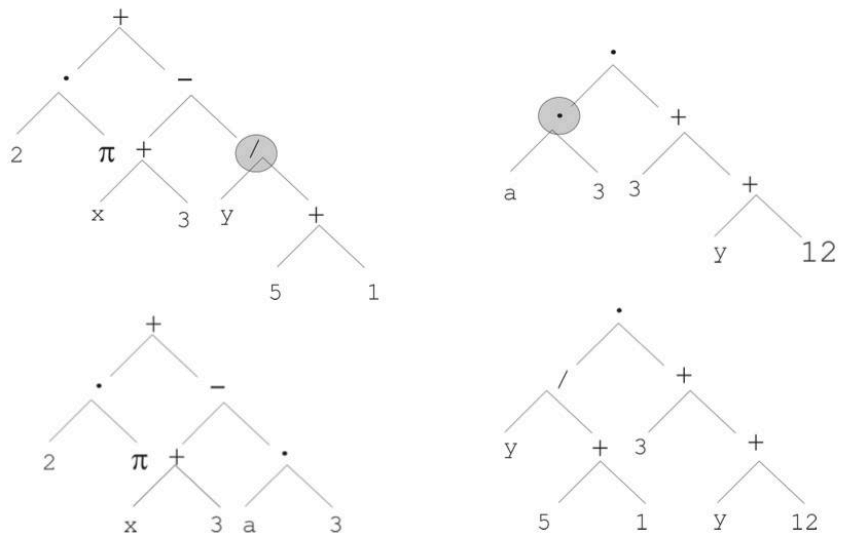
In the other approaches like GA and ES, genotype and phenotype were the same. In GP instead, we usually have to encode a formula in another representation because this parse tree can't be used. So, since it's not easy for us to evaluate the genotype, in GP we use the phenotype.

GP vs ES/EP

- In GP the parse-tree can become bigger because of the mutations (for example, appending parts to the formula), while in ES/EP strategies the structure was fixed (same length)
- GP has a non-linear structure because we are using a tree

GP operations

Mutation = In GP, a mutation operation can be anything like changing a node, swapping a node with another or expanding the tree. **Introduced in 92 with a low probability of 0.05** . At the first Koza version there was not mutation (Koza Biblio)
Recombination = In GP, recombination means randomly swapping: we create two new individuals as copies of the two parents and then we swap one subtree from each one.



Destructive Crossover → The **problem** with recombination is that it might destroy an individual's structure. This leads to a lot of exploration and never going in the direction of exploitation.
Solution: **LARGE POPULATION** → a very large population helps me to find individuals similar (parents similar to offspring) even if we are changing a lot the structure of the individual. This is because the step of mutation or recombination can change a lot the structure of the individual, and of closure if we have a small population, a big step has very different values (in terms of fitness), in a big population instead, the same big step, probably, obtain a more similar value.
Since in EA the offspring should have just small differences from the parent's structure, this is an issue. To avoid this problem, we can use **homologous crossover** which swaps only **similar** subtrees. Small changes are preferred.

Encapsulation → freezing the structure (schema) for an era (epoch) and changing only the parameter. If we are lucky we freeze an useful structure, and the algorithm work is simplified
We need a very large population

Other considerations

- How do we initialize the tree at the beginning?
    - Full, all trees have max depth → cannot grow anymore
    - Grow, trees randomly built during the generations
    - Ramped half-and-half (50% times is full, and 50% times are random values)
- **Schema Theorem** = In GP we can study trees as building blocks by looking at optimal subtrees in the context of our problem.

- **BLOAT =** Individuals become larger and larger, so the larger individuals will survive against the others. The tree becomes too large and, according to the fitness, we are doing good.
  The tree doesn't generalize the problem, so if we switch to a similar problem we don't get the same performance (overfitting).
    - One possible solution is to favor small individuals. Reduce operator that creates offspring larger than the parents (not working)
    - Operator that trim some structure that recognize that are unnecessary
    - Favor small individuals
    - Best solution: **fitness hole** → we compare the fitness, but in the fitness there is a hole (where we can put some other parameters to compare) which represents the size . So we can compare the size in a probability ex. 90% we are comparing the fitness and 10% the size. We can change the percentage of course. This approach can be used also in other strategies and it is perfect for tournament selection.
    - Mutation/Recombination should be limited so that we don't add a lot of new nodes
- **Introns =** Useless or redundant information that amplifies the genetic material without contributing to the final solution. Introns can contribute to BLOAT (fat individuals)
- GP is not efficient:
    - Parallelizing evaluation of individuals (on different pc)
    - Put GP alg. in GPU

Evolutionary Programming

Finite state machines (FSMs) represented individual organisms in a population of problem solvers.

A FSM is described by:

- Initial state (taken from a set of states S), and as the input is received the state is updated.
- When the input finishes the last state represents the output.
- The transition from one state to another is done through a *transition function*

An example of FSM: (if the number of zeros is even we return s0, otherwise we return s1)

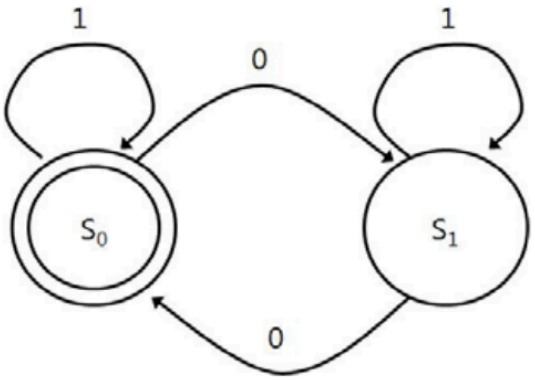


Figure 1. State Transition Diagram

EP is often applied to continuous optimization problems (the input is a sequence). The main objective is to predict the next value of a sequence (or a property of it) by evolving an FSM. An example is given a sequence from 1 to n, is n+1 prime or not?

In EP these are the main features:

- **No recombination**
- **No parent selection** → One point in the search space stands for a species, not for an individual and can not be crossover between species
- **Mutation**, done through different actions:
  - Add/Remove a state
  - Change transition function
  - Change initial state
  - Change output

Survival selection

Understand if this part is useful for the exam since he didn't explain it.
(u + u) → we take u elements from the population, we apply mutation on them, and then we do a tournament between the newly generated individuals and the parents. The bests will survive
Steady state is used as a stopping condition. If we can't improve anymore, we stop.
No elitism → The best individuals from the current generation are NOT directly copied to the next generation.

Offspring machines are created by randomly mutating the parents and are scored in a similar manner. Those machines that provide the greatest payoff are retained to become parents of the next generation, and the process iterates.

BenchMark

One Max (John Holland Problem) → The goal is to build a string of ones. Fitness is the sum of the elements . John Holland Problem
Knapsack problem
I need to maximize the value carried without exceeding the maximum weight of the backpack

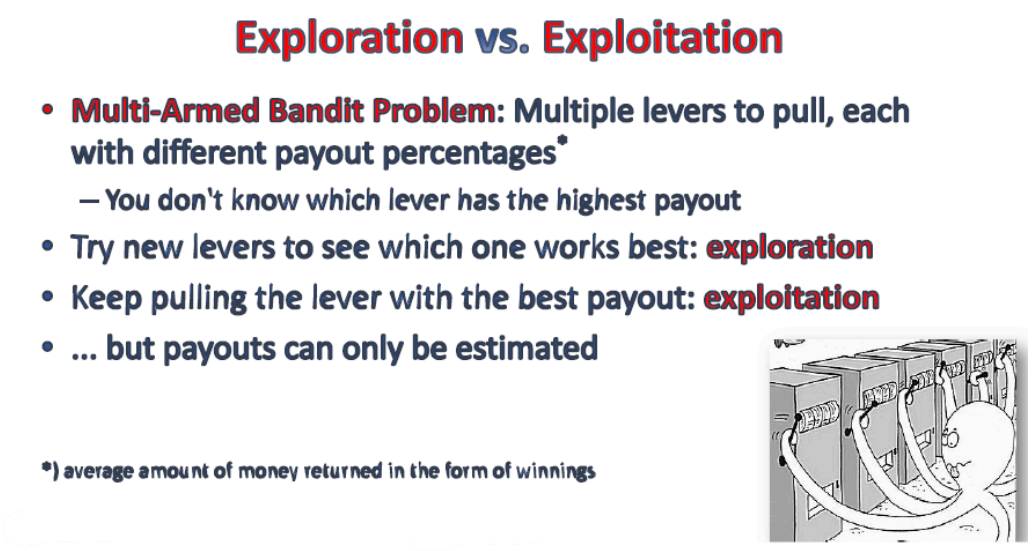
- **Multidimensional** → manage more than one constraints : weight limit / volume limit
- **MultiObjective** → target different goals, for example cost and sentimental value

Set Covering problem → Given a family of sets, find the best subfamily (of minimum cost) that cover all the area

Representation

We have already introduce some concepts about representation:

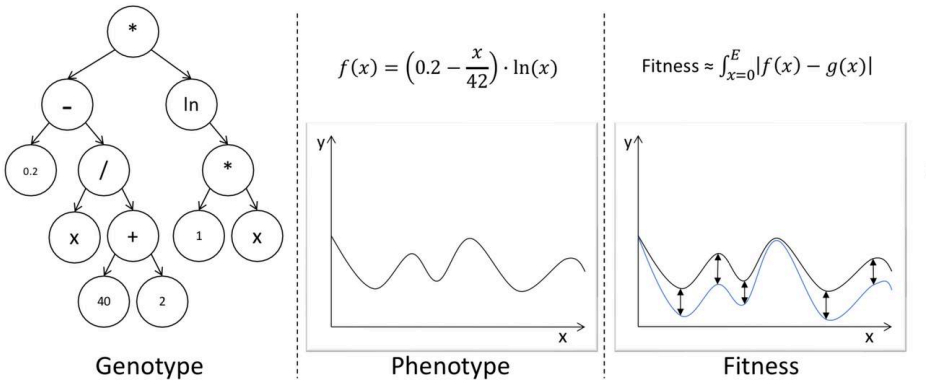
- **Genotype** = the internal representation of the individual (where the genetic operations are applied).
- **Phenotype** = the candidate solution that is encoded in the genotype.
    - In other words, the intermediate form in which the genotype needs to be transformed for *evaluating the fitness*
- **Fitness** = how well the candidate solution solves the problem. Fitness landscape shows the solution, not the problem.
    - **Fitness Landscape**: Shows the solution . it is impossible to represent as a linear function.
        - UniModal → Local optimum = global optimum
        - MultiModal → Many Local one Global
        - Isolated Fitness LandScape → small number of picks that creates basins of attraction, with a global optimum
        - Telephone pole →a peak in the center top and all around a valley. The goal is to catch that peak, but since it's very small, it's difficult.
        - Plateau →large region with the same fitness, nightmare. The algorithm doesn't know where to move. The algorithm seeks something new.
        - Noisy →very difficult to find the global optimum.



If the genotype can be directly evaluated then genotype and phenotype **coincide**.

Close Solutions → Two solutions are close if i can go from one to the other in one mutation

For example, an individual could represents a function f(x), the objective is to estimate a function *g(x)*



Thus we have an **indirect mapping** between genotype and the fitness value. We can also see that some genotypes can be mapped to the same phenotype (and some phenotypes can be mapped to the same fitness value).

Genetic operators:

- Mutation: small tweaks, **exploitation**
- Recombination: big changes, **exploration**

Since we are modifying the representation, we must ensure that the genetic operators won't create a structure that is not encodable at a phenotype level (and so it can't be evaluated).

Principle of locality

Small differences (in the genotype) bring small differences in fitness. Since we are using a mapping between genotype/phenotype/fitness we have to take into account this when designing the fitness. For example, the fitness should not have big differences when the genotype has only a small difference.

Discrete Representations

Binary representation = virtually we can encode anything (but we may lose some structures).

- **mutation** → bit flipping
- **recombination** → 1-cut crossover (generalized by n-cut crossover). Subject to **positional bias**, meaning that is more likely to keep together genes that are near while further genes will always be changed.

Bias example → American jails full of black people, so the algorithm thinked that black people are more likely to be imprisoned compared to white people (without considering other useful values). useless values are used like useful values.

Integer representation = everything is a number, so it's a wider representation.
We could introduce **Categorical** values (to categorize individuals, but for some situations the numbers will introduce a wrongful distance).

- **Mutation**
  - **random resetting** (pick randomly a value from the allele as a new value)

    - **creep mutation** (increase or decrease of the value, respecting the allele domain).
- **Recombination**
    - **1-cut crossover**
    - **n-cut crossover**
    - **uniform crossover** → for each gene select the value from one of the two parents according to a probability (solves the positional bias)

Permutation representation = a particular integer representation where the solution is stored using the order (aka the sequence) of the integers.

- Mutation = we don't change the values but we change the **order**.
    - **Swap** = swap two numbers.
    - **Scramble** = shuffle a set of genes.
    - **Insert** = shift a set of genes.
    - **Inversion** = a set of genes is inverted.
- Recombination = we want to inherit the order of a subset of the genes from one parent and the other from another

Continuous Representations

Floating Point representation = each number now is a floating point representation (not really continuous but close enough).

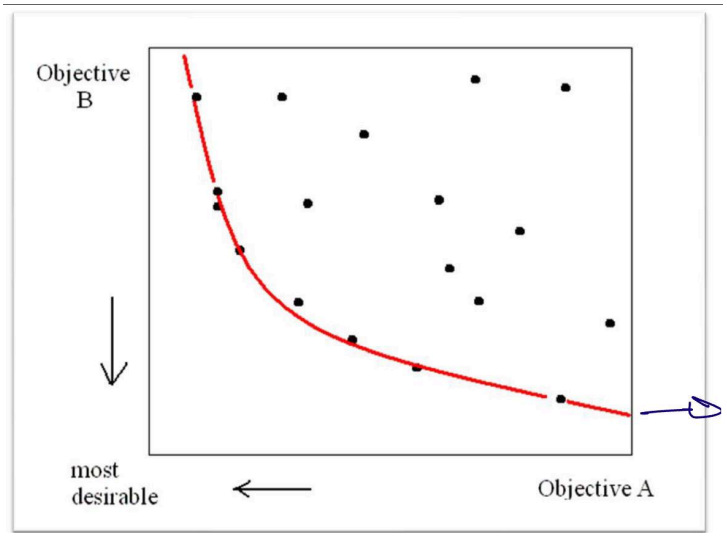
- Mutation = use a Gaussian distribution.
- Recombination = use uniform crossover or averaging crossover (for each take the average from the two parents).

Multi-Objective Optimization

Up to this points we have used fitness functions based on a single objective. We can generalize and include situations where fitness is a combination of different objectives. Note that we can have **conflicting goals**.

We can thus introduce **pareto front**:

- Any solution that improves an objective will always worsen another one.



Example:
A → Quality of our product
B → How much we spent for that product (the less the better)
It is impossible to reach the best in both the two parameters, of course if you spend more you will reach a better quality.
The red-line is the Pareto Front and it is the *limit of our solution*. We are creating a *Balance*

Only 3 situations:

- A solution is very good for B but very bad for A.
- A solution is very good for A but very bad for B.
- The solution is in the between for both.

Diversity Promotion

Tendency of an algorithm to converge towards a point where it was not supposed to converge to in the first place.
EAs general inability to exploit environmental **niches**.

Exploration vs Exploitation

Recombination mixes together two or more solutions to create offsprings → **exploration**
Mutation performs change in an individual → **exploitation**
But when parents are all similar, the effectiveness of recombination is limited.
Increasing diversity should be generally beneficial

Niches → Subspace in the environment with a finite amount of physical resources that can support different types of life.
Niches favor the divergence of character.
Niching means grouping similar individual

- similar spatial positions (island model)
- similar genotypes

- similar phenotypes

Diversity and How measure it

Diversity → distance metric: how far the individual is
- from a subset of the whole population
- from a single individual

This **diversity is measured** at level of
- **Phenotype** : Usually ad-hoc
- **Genotype** : Different genotypes in population / GP subtree frequency / edit distance (Levenshtein distance → minimum number of operations to transform a string into another) / entropy and energy
- **Fitness** : Usually trivial (banale / insignificante)

Different fitness values imply different phenotypes, different phenotypes imply different genotypes

How diversity is promoted

Promoting diversity has often been seen as the key factor to improve performance.
- Fitness scaling
- fitness holes
- tweaking selection mechanism
- adding selection mechanism
- multiple populations
- population topologies

A methodology for **promoting diversity alters the selection probability of individuals**
When selecting an individual, the probability of that individual to be selected is adjusted by a factor that takes into consideration the diversity of the population (as a function of population and individual)

$$\bar{p}_{x|\Psi} = p_{x|\Psi} \cdot \xi(x, \Psi)$$

The individual *x* is selected from the population set *psi,* corrected by a corrective factor.

We can implement a diversity factor by using:
1. **lineage (LIN)** → takes the history of the individual into consideration. In this case the *correction factor* does not depend on the individual structure
2. **phenotype (PHE)** → we consider the result of the fitness function. It is usually impractical because it can happen that two very different genotypes have the same fitness values, so they are wrongly considered similar.

3. **genotype (GEN)** → here we use the information encoded of the individual. In this case, we need a way to measure how closer the genomes are (**task-dependent**)

Type of selection

In ES there are two moments where we have to use selection:

1) **Parent selection**, usually it is done randomly (*non-deterministic* process)
2) **Survival selection**, it is *deterministic* (based on fitness for example)

Methodologies

In the next sections there are a few methodologies used to introduce or promote diversity. All of these diversity promotion methodologies will be used for both types of selection: parent selection and survival selection.

Island model

In this model, we partition the population into subpopulations called islands. Only local interaction between individuals in the same island are allowed. Since the belonging of the individual to a certain island depends on its parent, this is a lineage-based model.
To introduce diversity, we can periodically move individuals between different islands. These individuals are the *migrants.* Even with migrants, this methodology might lack useful global interactions.

Segregation model

In this model the population is partitioned again into N sub-populations like in the island model, with only local interactions allowed. The difference here is that, when we have stagnation, the N sub-populations are merged into N-1 subpopulations.

The selective pressure decreases during the evolution (it becomes easier for the individuals to survive as the evolution process goes because in each set the size of the population is increased thank to the merge)

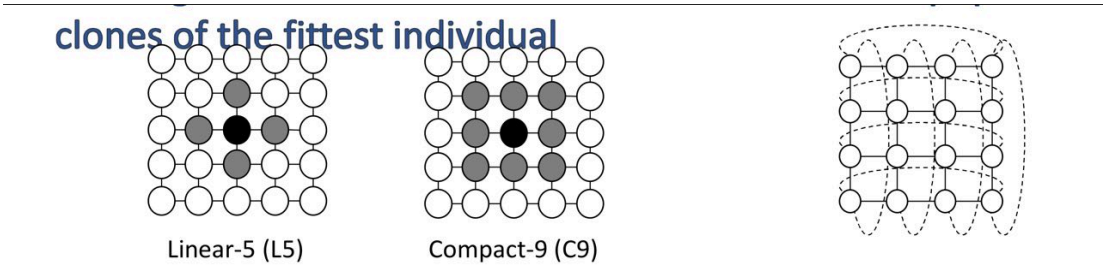
Note: stagnation means that we are stuck in a certain area → fitness not improving

Hierarchical fair competition model

In this model the population is partitioned into N sub-populations with similar fitness, so this is a phenotype-based model. Only local interactions are allowed and the individual offspring is promoted/demoted to a different population based on their fitness value.
The practical problem with this model is that newborns (which are periodically randomly put in the population) may experience reduced competition because they are put at the bottom of the hierarchy.

Cellular EA model

In this model we have a more strict interaction policy: we create a fixed structure called *lattice* (a fixed topology) and in this topology only local interactions between the neighboring individuals are allowed. Since we classify this model as being lineage-based, the new offspring that perform better than the parent will replace them in the topology.



For example, using the first topology model (leftmost) you can see that the individual at center (black) can only interact with the gray individuals.
By limiting the interactions between the individuals considering only the neighbors, we avoid the takeover of the population by clones of the fittest individual.

Deterministic crowding model

In this model offspring compete against parents for survival. The idea used is called **implicit neighborhood**, meaning that we can assume that parents and offsprings are similar and there is no need to measure the similarity.

Allopatric selection

This model is typically used when we have genetic operators that create large offsprings and the competition for survival is done between the whole set of new offsprings. Using this model we avoid problems related to the replacements of the population by the newly generated offsprings.

Fitness Sharing model

This diversity promotion model involves the usage of distance metric between individuals, so this is why we can classify this model as genotype-based. The idea here is that after calculating the fitness of the individual, we scale it down by the distance metric factor that depends on the distance between the individuals.

Using this model we use the idea of **explicit neighborhood** meaning that we explicitly define a distance metric between the individuals. Moreover, by scaling the fitness function we reduce the attractiveness of densely populated area.

Clearing model

In this model we create niches based on the genotype and the *best k-individuals* in a certain niche will survive while the rest will die (fitness equal to zero). This clearing strategy allows the creation of densely populated areas.

Standard crowding

In this model, all the new individuals replace the most similar individual in a random niche of size CF. This is also a genotype-based model because the niches depend on the genotype. This model is very similar to the generational approach (meaning that new individuals directly replace old individuals).

Crowded-comparison operator model

In this phenotype-based model, we favor solutions located in less crowded regions of the solution space. It requires a strong correlation between phenotype and fitness.
If you take a group of individuals with the same fitness, it means that the phenotype (hence the genotype) must be similar.

In this model we don't create specific neighbors but there is a sort of artificial niche introduced by the idea of free territory around an individual solution.

Restricted Tournament Selection model

In this model you let the new individuals compete with the most similar individual in a random niche of a given size CF.

Sequential Niching

The most promising individuals in the search space after each run are altered so as to become less interesting in future executions. The idea is to avoid over-exploitation.

Adversarial Search

In adversarial search, we study games with the objective of defining an **optimal policy**. A policy is a function that, given a state, returns an action. Put another way, we can say that we want to find the optimal move (**optimal ply**) for a given player.

Games can be classified in different ways:

- **Deterministic** vs **Probabilistic** games
- **Perfect** vs **Imperfect** information:
- **Turn-based** vs **Real-time**
- **Zero-sum** vs **Non-zero sum**

For example, chess is a turn-based, zero-sum, deterministic game with perfect information.

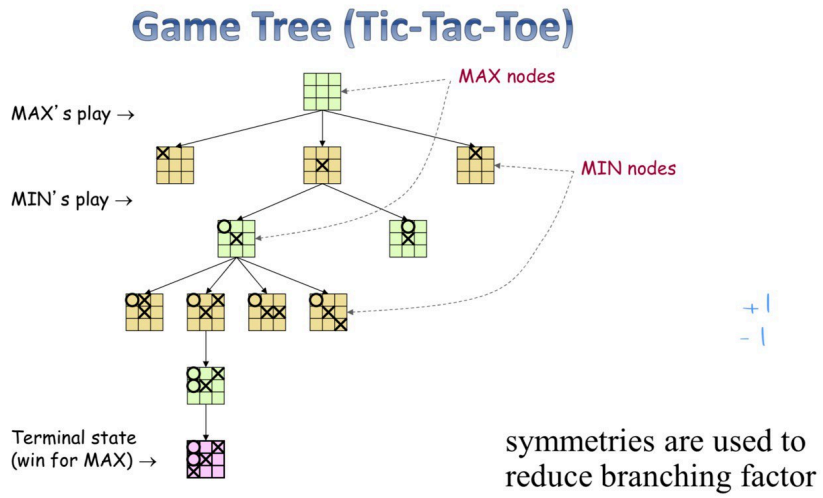
Chess is a perfect-information game because each player can see all the pieces on the board.
Poker instead is an imperfect game, because you cannot see your opponent's move.

Adversarial search is a bit different from a regular search problem because we have an opponent and we cannot predict his actions during the game.
So, for this problem, the solution is not a fixed list of actions but a **policy** which is a mapping from a state to the best move in that state.
Efficiency is critical to playing well.

Deterministic Games

MinMax → Von Neuman then rediscovered by Alan Turing

MinMax is a decision rule for **minimizing the possible loss** of the worst case. This doesn't mean that we are maximizing the winning. As an assumption, we can say that this model is used in deterministic, two-player games with perfect information.
MinMax is complete only if the tree is finite.
Each player maximizes their own utility at their node. Then Utilities get propagates (backed up) from children to parents



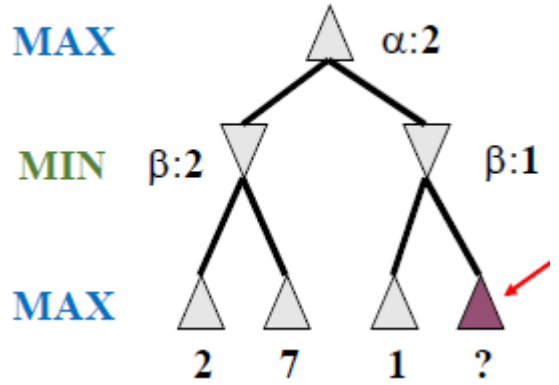
The picture above shows an implementation of MinMax in Tic Tac Toe.

- ➔ Our opponent is called MIN player because he will always choose the action that is better for him, hence minimizing our reward.
- ➔ We are the MAX player because we want to choose the action that will lead us to the maximum of our possibilities.
- ➔ Since the opponent will choose the worst results for us we are maximizing the worst results.

We have some properties of minmax:

- It's complete if we are able to explore the entire tree.
- It's optimal against an optimal opponent.

Problem: a lot of games have a state space which is huge (i.e. chess is 10^120), so exploring the entire search space would be impossible.
An improvement to MinMax is called **Alpha-Beta pruning**. We can understand it by using the following example:



- We have already evaluated the left sub-tree, getting a final result equal to 2.
- For the right sub-tree, we don't actually need to completely explore it. Why?
    - Since it's the MIN turns we already know it will choose a value which is less or equal to 1.
    - But then the MAX player will choose the maximum between 2 and a value which is less or equal to 1, and it will always choose 2.
    - Hence we can already compute the final value of alpha without having to explore the entire tree because we already got the maximum.

If we need more optimization (the search space is still too large) we can use **hard cut off**, simply limit the search to a certain depth. This will lead to what's known as the **Horizon effect**: since we are stopping too early some events that are beyond the depth limit won't be considered.
For example in our implementation of Quixo, when we use "Max Depth = 3" → If at the 4 layer we would win, the minmax will never consider that case

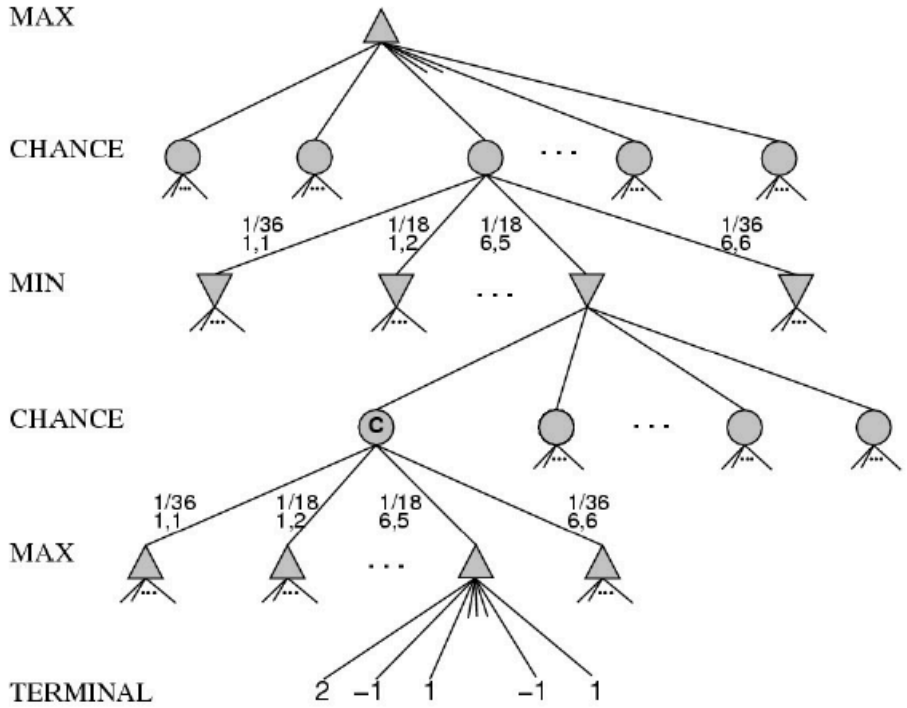
There can be strategies to avoid the horizon effect, caused by cut-off. **Those strategies basically increase the depth limit only for certain subtrees** if the state or move can be considered interesting to explore.
→ The evaluation function if a state/move is interesting must be defined by us.
Singular Extension → Increase depth when evaluating volatile positions
Quiescence Search → increase depth with brute force searching on selected singular moves (moves that return a value much better than all their alternatives)

Hash Table → to store previously expanded states

Stochastic Games

In these type of games, differently from deterministic games, we have to take into account a certain probability *p* after each action. In this context the action doesn't directly trigger a transition to a new state; the probability must be known.

In the case of MinMax using a Stochastic strategy we need to take into account the probability between the actions.



We can see that in between Mix and Max players we are considering a **probability for transitioning from an action to a new state.**

Considering the MinMax standard formula, here the reward changes because of the probability.
In fact, the MIN player will take the action that, on **average**, will result with a lower reward:

$$Value(node) = \max_{my\ moves}\left(\min_{opponet's\ moves}(\mathbb{E}[Reward])\right)$$

$$\mathbb{E}[Reward] = \sum_{outcomes} Probability(outcome) \times Reward(outcome)$$

Average reward is simply computed for all possible outcomes as the **probability of the outcome multiplied the reward.**
Like the standard MinMax, we can consider all the methods like Alpha-Beta pruning, Quiescence search and singular moves.
But we need to consider that now we need to compute an additional layer for taking into consideration the average value, so the computational complexity is higher compared to deterministic games.

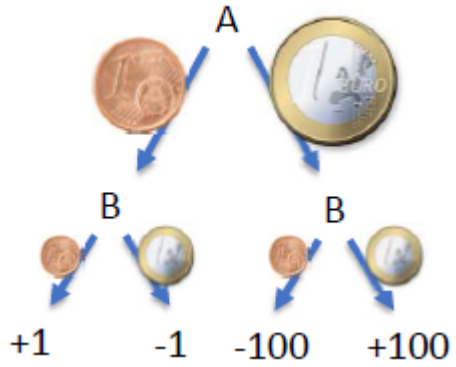
Games of imperfect Informations

When there are imperfect information (so we don't know some parts of the states) we have two strategies:

1. Treat unknown as a state which is always the best for our opponent → Deterministic
2. Treat unknown as random considering all the possible outcomes → Stochastic

Let's at an example:

- Alice (opponent) chooses a coin.
- Bob (us) calls "euro" or "cent":
    - if correct we win +1 or +100 based on which coin
    - if wrong we lose -1 or -100 based on the coin.



⇒ MinMax: let's minimize our loss, but let's consider the two cases:

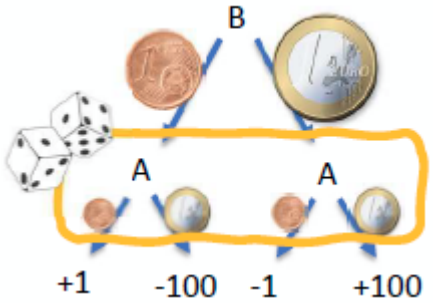
In the deterministic case, we have perfect information and we assume that the coin will always be favorable to Alice.

So we should pick the cent.

In the Stochastic case instead, we pick an action and we compute the probability of choosing that specific action. In the specific example above, we have:

- If Bob calls Cent: ½ * 1 - ½ * 100 = -49.5
- If Bob calls Euro: ½*100 - ½ * 1 = 49.5

Since 49.5 is the max, Bob will choose Euro (49.5).



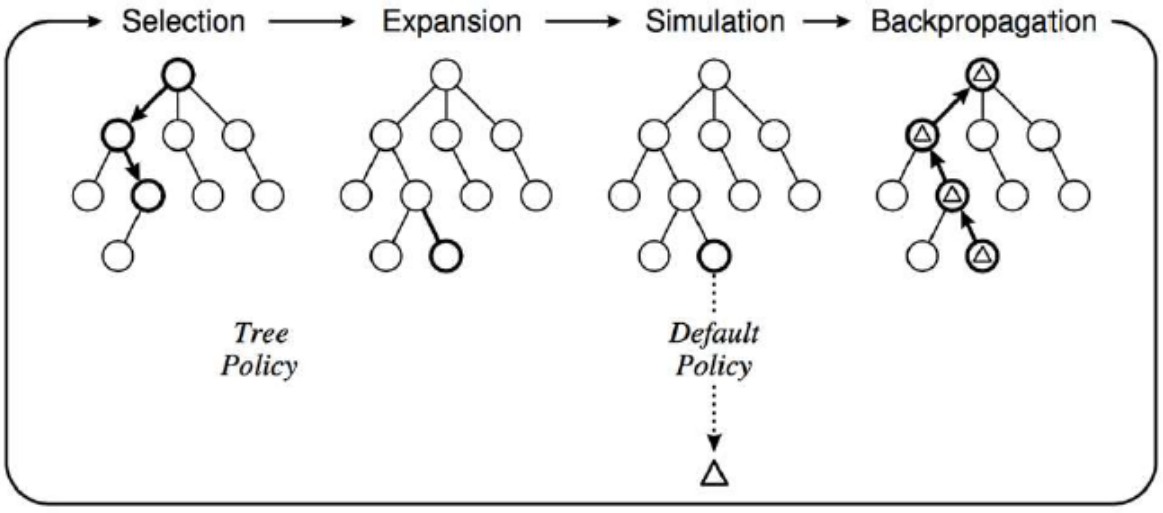
In this case, compared to the determinist model, we got a different outcome. In fact, Bob will here choose Euro instead of Cent.
This is in general a problem of imperfect information: modeling the unknown as a probability or a “pessimistic scenario” can yield different results.

Stochastic Search - Monte Carlo Tree Search

MinMax trees can result in a large search space that is unfeasible to compute. For this reason we introduce **Monte Carlo Simulation**.
Monte Carlo Tree Search

- **Tree policy** : select a leaf node for expansion (trade off from exploration exploitation)
- **Default Policy** : simulate (ex. random moves) until a terminal state is reached
- **Back-propagation** : back propagation the outcome to update the value estimates of internal tree nodes

The idea behind Monte Carlo Simulation is: instead of evaluating a state by computing the entire **subtree** we **sample some simulations** (for example with random moves) of the game and then we compute the expected reward based on the average of the results of those simulations run using MinMax.



$$ExpectedReward \approx \frac{1}{n}\sum_{i=1}^{n} Reward(random\ game)$$

Reinforcement Learning

The idea of Reinforcement Learning is modeling the problem as an agent that learns to make decisions by interacting with its environment. We need a set of elements to formalize the problem:

1. **Environment** → The external system with which the agent interact
2. **States,** set of states S
3. **Actions,** set of actions A
4. **Rewards,** set of rewards R

The goal is to maximize the **sum** of the rewards.
For each time step *t* , the agent:

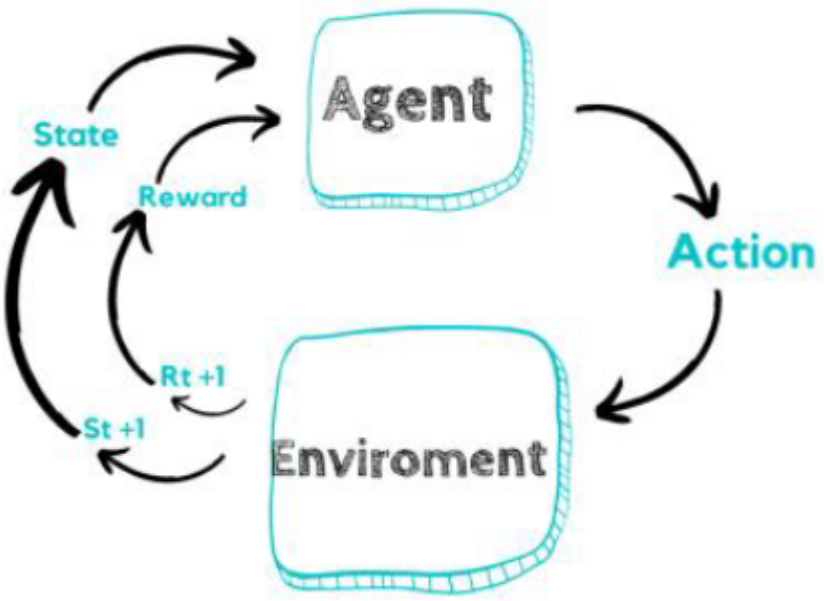
- receives a representation of the state *S_t* belonging to set of states S
- selects an action *A_t* belonging to the set of actions A
- gets a reward *R_t* belonging to R defined by the reward function

We can represent the **trajectory** (history) that represents the process as a sequence of *S_t,A_t,R_(t+1)*

Expected rewards becomes:

$$G_t = \sum_{u=t}^{T} R_{u+1}$$

which is the sum of all the rewards
Goal of the agent → maximize the expected return of rewards



Episodic task = at some point we know the simulation will finish (example, tic tac toe)
Continuing task = no guarantees that the simulation will end

In the case of continuing tasks, at some point the importance of the reward starts getting lower and lower. We need, after some time steps, a way to discard those lower rewards and this is called the **discount rate**.

Thus, if we get very high reward but we have spent a lot of time in the simulation we don't consider that as import as immediately get high rewards

Expected Discounted Return of Rewards at time t:

$$\cdot \overline{G}_t = \sum_{k=1}^{\infty} \gamma^k R_{t+k}$$

Since *gamma* is < 1, so at every step it becomes lower and lower. Gamma is the "discount rate" . The first reward are more important than the subsequent ones
We can see that the lower the gamma value the quicker the discount rate will get to zero and the greedier we are (quick but not precise result).
This is now an expected **discounted** reward.
Goal of the agent → maximize the expected DISCOUNTED return of rewards

Reward Hypothesis

⇒ **All the goals can be described by the maximization of expected cumulative reward.**

We can also introduce the idea of **delaying the reward**: it is okay if a certain action doesn't have an immediate reward, because we are interested in gaining a long-term reward which is bigger.
Action may have **long term consequences**.

An example with two game of chess:

- In one game we play only to capture some pawns (reward=1) but at the end it's a draw. History = [1, 1, 1, 1, …, 0 (final move)]
- In the other one we play some strategically without capturing a lot but we win (final reward = 100). History = [0, 0, 0, …, 100]

We can see that the expected reward for the second strategy is higher compared to the first one.

State : A state is the current situation of the environment at a certain time. If we look at the Trajectory until the state **S_t** and action **A_t** we can derive the new state **S_t+1**.

We have to separate between two types of state:

- **environment state**: true state of the environment. It might be not fully observable
- **agent state:** agent's internal representation of the environment.

Observability
If the environment is **fully observable** (aka complete information) the two states are the same. If the agent has a **partial observation** of the world the agent state will be a subset of the environment state (like in the game of poker we only know our card, not everyone's cards).

Markov State

Another particular state that we will use in the future is the **Markov state**, called in this way because we use the Markov assumption:

➔ *The next state depends only on the current state and not on the entire history*

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_0, S_1, \dots, S_t]$$

An example is Game of Goose (gioco dell'oca): for computing the probability for the next state we only need the current state, we don't need to look at the entire history

Agent's Policy

Agent's policy → it is the definition of the agent's behavior.

- **Deterministic policy** : Action = policy(state)
- **Stochastic policy** : Prob(A_t | S_t)

The **policies** are a mapping between state and action.
For computing these policies we need to calculate an expected reward on our decision, given a state.
if we have a function that given a state, it returns a reward, the policy is to choose the state with the highest reward.

Policies

When speaking about policies, formally we say that an agent "follows a policy." For example, if an agent follows policy π at time t, then $\pi(a|s)$ is the probability that $A_t = a$ if $S_t = s$. This means that, at time t, under policy π, the probability of taking action a in state s is $\pi(a|s)$.

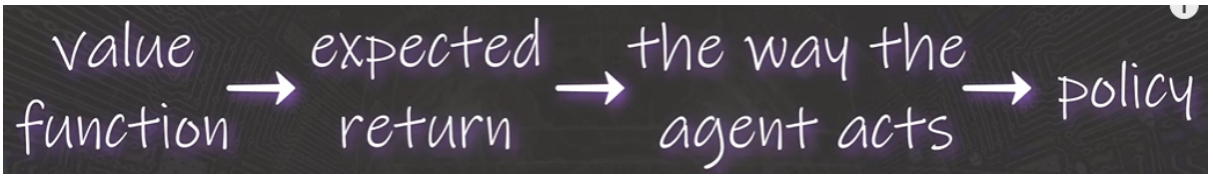
Note that, for each state $s \in \boldsymbol{S}$, π is a probability distribution over $a \in \boldsymbol{A}(s)$.

Agent's Policy

- The definition of the agent's behavior
  - A *map* from state to action
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

A goal here is to Optimize policy to Maximize future rewards

State-value functions (not considering the action)



We have to define the behavior of an agent using policies that are basically mappings from state to actions.

State Value Function

We call **state-value functions** for a given policy the average expected reward starting from state ***s*** and following the **policy** until the end.
Before choosing an action, thanks to these state-value functions, we know beforehand what happens if we follow a specific policy.

We discover the expected future reward of being in a state S following the Policy P

$$v_\pi(s) = \mathbb{E}_\pi[G_n|S_t = s] = \mathbb{E}_\pi\left[\sum_{k=1}^{\infty} \gamma^k R_{t+k} \middle| S_t = s\right]$$

We can see that this is computed as the average **expected reward** starting from **s** and using the **policy** π.

v_π(s) → state value function

Action-Value functions

Action-value function

Similarly, the *action-value function* for policy π, denoted as q_π, tells us how good it is for the agent to take any given action from a given state while following following policy π. In other words, it gives us *the value of an action* under π.

Formally, the value of action a in state s under policy π is the expected return from starting from state s at time t, taking action a, and following policy π thereafter. Mathematically, we define $q_\pi(s, a)$ as

Q-value

$$q_\pi(s, a) = E[G_t \mid S_t = s, A_t = a]$$

Q-function

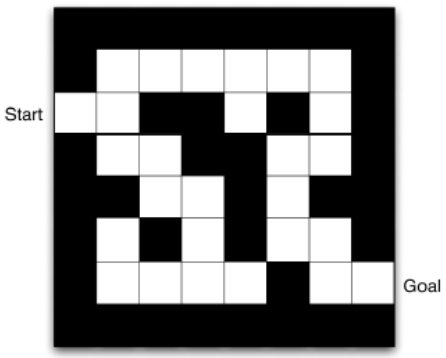
$$= E\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right].$$

The action-value function return the **expected average reward** starting from a state ***s*** performing action ***a*** and following the given policy π until the end.
q is named Q-function
It is similar to the state-value function but in this case we assume we have the current action.

$$q_\pi(s, a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} \middle| S_t = s, A_t = a\right]$$

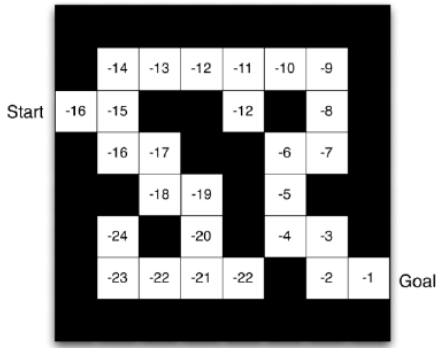
We can thus define a policy from a value function:

- ➔ Given a state *s* we can now compute the state-value function for each new state given an action or we can directly compute the expected reward using the current state and each action.
- ➔ Choose the action that maximizes the expected reward.

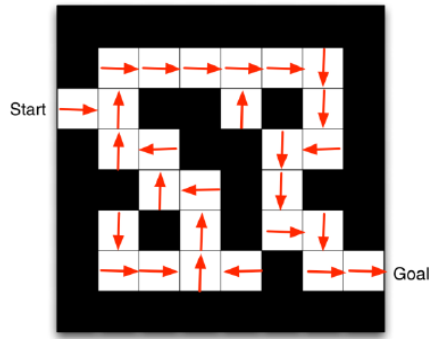


Let's look at this example:

Imagine that an agent has to go from start to the goal by exploring the maze. (maze = labirinto)



We can imagine that we have already computed a state-value function.
In this case it's the distance from the current positions and the goal.



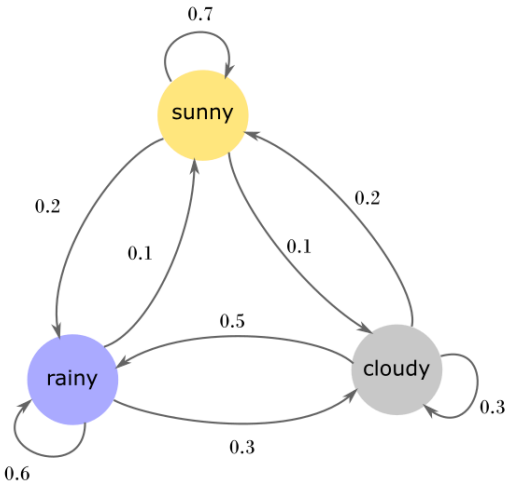
Now for each position we can decide the next one by simply moving to the position that maximizes the reward (in this case we are getting closer to the goal position).

Markov Processes

A markov process is a sequence of random states *S_0,...,S_n* with the **markov property of being memoryless**.

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_0, S_1, \dots, S_t]$$

Having the entire trajectory (history) or having only the last state make no difference
We can see that the probability of moving from Sunny to Cloudy is always 0.1 (10%), no matter the weather of the previous days.



For computing the reward we can use the same function as before (including also the discount factor *gamma*).

Most Reinforcement Learning problems can be formalized as **Markov Decision Processes**, where we have a value function that follows this form:

$$v(s) = \mathbb{E}[G_t | S_t = s]$$

So we compute the value function in *state s* by computing the average of the *Expected Discounted Return of rewards* at time *t*, given that the state at *time t* is equal to *s*.

Markov Decision process

A **Markov Decision Process** is a stochastic decision-making tool based on an environment where we have full observability and markov hypothesis (the current state *s* completely characterizes the process).
It returns a State transition probability.
Given a State S, and an Action, it return the probability that my new state will be S'
P(s' | s, a)

Agent has the goal of maximizing the cumulative reward

Agent-Environment Interaction

Since we are talking about stochastic processes, we now introduce **stochastic policies**. A *stochastic policy* is a policy that maps each state to a probability distribution over actions. The advantage of a stochastic policy is that **it can capture the uncertainty in the environment.** For example, in a poker game, the agent may not always take the same action in response to the same hand since there is a probability of winning or losing depending on the opponent's hand and how the betting has proceeded. In such cases, a stochastic policy learns the best strategy based on the probability of winning.

At each time step:

- The agent receives a representation of the environment state S_t in S
- Select an action A_t in the set A(S_t) of available actions given that state

One step later:

- The agent receives a reward Rt+1 and finds itself in a new state S_t+1

The agent implements a stochastic policy : a mapping from states to probabilities of selecting each possible action
The **policy** p(a|s) is the **probability of taking action A when in state S**

Stochastic Policy → A policy is a Stochastic rule by which the agent selects actions as a function of states.

Reinforcement learning methods specify how the agent **changes its policy using its experience**.

Action Selection

- At time t, the agent tries to select an action to **maximize** the sum G_t of discounted rewards received in the future

$$G_t = R_{t+1} + \gamma + \gamma^2 R_2 + \cdots + \gamma^{T-t+1} R_{t+k+1} = \sum_{k=0}^{T} \gamma^k R_{t+k+1}$$

- Given current state s and action a in that state, the probability of the next state **s'** and the next reward **r** is given by

$$p(s', r|s, a) = P_r(S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a)$$

Bellman Equation

The idea behind the bellman equation is: The reward in a given action is equal to the **reward** from the given action ***a*** plus the average **reward** from **future** actions (successor states).

- The equation express the relationship between the value of A state S and the values of its successor states
- The value of the next state must equal the discounted value of the expected next state, plus the reward expected along the way

$$v(s) = \mathbb{E}[G_t|S_t = s]$$
$$= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1})|S_t = s]$$

Comparing different Policies

A policy is defined to be better than another policy if its expected return is greater than the one of the other policy for all states

Q-Learning

A model is a mathematical way to define an environment and allows making inferences about how the environment will behave. Some strategies like the one we saw before are **model-based** approaches, but there are also **model-free** approaches where we learn exclusively from trial-and-error, hence there is no modeling of the environment.

While on a model-based approach the estimation includes a probability from S to S' given by a model of the environment:

$$Q(s,a) = \sum_{s'} P(s' \mid s,a)[R(s,a,s') + \gamma \max_{a'} Q(s',a')]$$

Q(s,a) : Action value function [Model based]
In a Model Based you are given (ti viene dato) the probability of S' given S and A

On the other hand for **Q_learning there is no model hence no probability**.
The update is done as we explore the states:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

We create an Action Value function that should converge to the real average reward.

- ➔ **Our policy** will be *"we can explore only some pieces of possible solutions, so we **follow the value (in the tree of states) with the highest reward**"*

In this case (Q learning) we don't have the **probability**, so the approach is trial and error

Q-Learning is a **model-free reinforcement learning** algorithm to estimate the quality of action in a given state with the idea of finding an optimal policy π^*.

The Q table represents the states with the associated actions.
The Q_func is updated in the way explained in the formula before. So we sum the difference multiplayer by a discounting factor (gamma)
Goal of the agent → maximize the expected return
Given some experience following a policy π, update the estimated value of that π (So we modify the policy, and that policy gives us an expected reward) for non-terminal states occurring in that simulation and then:

- wait until *t+1* to update the *V(S_t)*
- learn from the partial returns

example
if after ten times that we pass in a certain state X, we win the game, in Q learning we are free to say that the expected reward for state X is very high.
If at the Q function I pass X, the reward is high.
This is not the same as computing a probability like in the classical model
Problem with Q learning is that it needs an enormous quantity of data
Q table becomes too big .
Neural Network solves this problem , they approximate the Q learning → Deep Q learning

LCS → a way to approximate Q learning by putting together different things, having a compact view of the Q table. Problem : they are very slow
There are some rules that are useful in different problem

Knowledge-based agent

In knowledge-based agents we use a knowledge based approach and an inference system to build new knowledge. A knowledge base is a set of representations of facts of the world.

Example: if the **knowledge base** is
- "*When it's raining Farisan brings an umbrella*"
- "*It's raining*"

Then we have a new knowledge: "Farisan has brought an umbrella".
Based on the knowledge base, the agent chooses an action.

KB → It is an amount of facts that we know. And we know a certain amount of relationship between these facts and some implications.

Similarly to RL, in Knowledge-based agents, at the beginning the agents know nothing about the environment. As the agent starts interacting with the environment it starts building a knowledge base that will be used to act (choose an action).
The more expanded and precise the Knowledge-Base is the actions that we can perform are better.

Entailment means that one thing follows from another, meaning that a certain knowledge base KB entails a sentence α. In practice every situation where KB is true also α must be true.

$$KB \models \alpha$$

Models are a formal representation of structured worlds.
We say that *m* is a model (*m* is an assignment of values) of a sentence α if α is true in m :
m = {"outside is raining" → true, "Farisan has an umbrella" → false"}
m' = {"outside is raining" → false, "Farisan has an umbrella" → false"}

We can define M(α) as the set of all models that make α true.
If α is *When it's raining Farisan brings an umbrella* the model
m = {"outside is raining" → true, "Farisan has an umbrella" → false"}
doesn't belong to M(α).

We can thus define entitlement as:

$$KB \models \alpha \text{ iff } M(KB) \subseteq M(\alpha)$$

If α is true we don't care that the knowledge base is true. We are only interested in verifying that when the knowledge base is true we can correctly inference that also α is true.

In practice, we can't work with entailment because we should build all the possible models and then check the subset relationship according to the sentence α. In practice, we use an

inference system which is just a set of rules that we can use to build up a proof that α is true given that the KB is true.

It exist a proof that starting from the hypothesis KB we can reach α

An example: we can use a logical system to model the example from before so:

- "*When it's raining Farisan brings an umbrella*" can be modeled using the logical implication: "*it's raining" ⇒ "Farisan brings an umbrella*"
- The implication rule says that if I have proof that is raining I also have proof that Farisan will bring an umbrella.
- We don't check the models, but we take it for granted since we are using an inference system.

Soundness = what we derive is actually true in the real word: ⊦ ⇒ ⊨
Completeness = everything that is true in the real world can be derived by our system: ⊨ ⇒ ⊦

Multi-agent systems

We use multi-agent systems when we have problems requiring massive computational resources or problems that are multi-agent by definition (for example,a country that should face an election: we have a lot of people/agents voting).

Centralized learning = In centralized training, agents share the collected experiences and learn from them together.
Distributed learning = Each agent collects its own set of experiences during the episodes and learns independently from those experiences.

We can make a distinction between Multi-Agent Systems:
Homogeneous agent types = all the agent are the same (for example, they use the same policy)
Heterogeneous agent types = the agents are different (for example, we could have one agent that uses RL and another one that uses a MinMax strategy)

Another distinction can be made considering the goal:

- *Common* goal or *different* goal for each agent?
- The agent must *cooperate* or *compete* against each other.
- Do they *share* a plan to reach the goal in the cooperative scenario?

There can be a common Knowledge shared by agents:

- **Local =** the knowledge is for each agent
- **Complete =** all the knowledge is shared so each of them can see the full scenario (for example Tic Tac Toe)

Based on the scenario we want to model the interactions between agents depending on multiple factors, like local or global communication, network topology (who can communicate and with whom), Push or Pull mechanism etc…
Agents can exchange information by *sending/receiving assertions* or *sending/receiving queries*.

Swarm Intelligence

A centralized, homogeneous and cooperative scenario. The idea of Swarm Intelligence is to have a **set of agents exhibiting a collective behavior**. The agents can contribute to and benefit from the group, can recognize, communicate and interact with each other. Each agent follows simple rules.

A real swarm example might be crowd (swarm of human agents) or city traffic (swarm of cars as agents).

Particle Swarm Optimization

We are in the context of **parameter optimization**, so we want to maximize a fitness function.
PSO is an improved version of Hill Climber based on using a swarm.

Each agent moves in a direction and tries to find a sort of a best local value (***pbest***) for that agent.
The agents then share the individual local max to find a collective best (***gbest***). This strategy tries to imitate human or insect social behavior by allowing the agents to learn from their own experience, sharing with each other and moving towards the goal.

We don't want everybody to converge to the **gbest** because in that case we will obtain only a local optimum.
We want a situation where each agent tries to do better than his **pbest** without reaching the same value of the **gbest.**

$$v_i = v_i + c_1 \times rand() \times (pbestx_i - presentx_i) + c_2 \times rand() \times (gbestx - presentx_i)$$

$$where\ 0 < rand() < 1$$

$$presentx_i = presentx_i + (v_i \times \Delta t)$$

For modeling the movement we use the velocity vector Vi.

Vi is a linear combination of the old velocity, the **pbest**, and the **gbest.**
We want to find a new gbest hoping to reach the actual local optimum.
The difference between this and Hill climbers is that here we are in a Swarm (not alone) and the individuals exchange information together.
Using the coordinates of **pbest** and **gbest**, each agent can then update its new velocity.

Ant Colony Optimization

This algorithm is an application of swarm intelligence for path search.
We start from a point and we want to reach the food.
There are many paths for reaching the food.
Every time an Ant tries to reach the food, it spreads a “pheromone” that it depends on the path that the ant has chosen.
The shorter the path, the higher is the pheromone, so the probability of another ant to choose the path with the higher level of pheromone increases if the path is shorter.
In this way, the stupid path will be eliminated and the better path will be explored from the beginning.

MetaHeuristic → trasformazione dal feromone a un metodo informatico che ci permetta di seguire la strada corretta più corta
All’inizio non funzionava
Important : viene aggiunta una traccia che svanisce lentamente, la probabilità aumenta che una formica vada nella direzione dipende dal feromone che c’è sul percorso
Balancing tra exploration (seguire una traccia di feromone più leggera) e exploitation
Best solution → max amount of pheromone
Usato per risolvere routing problem

Artificial Bee Colony

different type of agent
Different types of bee that communicate in some way
Some bee are creating the environment
Several variants
Formula → inutile

Differential Evolution → parameters optimization

Algoritmo che funziona veramente
Funziona ma non sappiamo il perchè
Prendiamo 3 elementi diversi da 3 punti nel search space
Aggiorniamo il primo con la differenza degli altri due
Usa il secondo e il terzo per esprimere una direzione e applica questa direzione al primo punto
y (offspring) = A + (B-C)

Estimation of Distribution Algorithm

Instead of storing explicit individuals, we generate an individual whenever we need it using distribution of probability
We can generate them whenever we want, and we have potentially infinite number of individuals
(idea 93 / 94)
Generate a set of individual (for example generating random number and if this random number is above a threshold, it is zero otherwise is one)
→ evaluate the generated individual
→ use the result of evaluation to update the statistical representation of the population
→ and then after sometimes we extract the champion

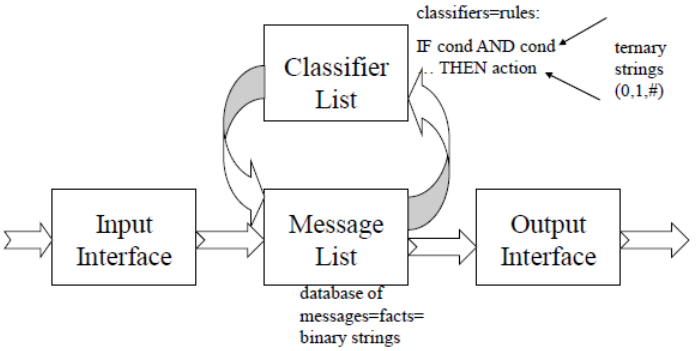
Fuzzy Logic
Something is not only inside a set or outside but is something in the middle.
It can be expressed with a probability between 0 and 1

se stiamo lavorando con un agent, ci permette di interpretare dei set di condizioni che stanno triggering l'agent possiamo interpretare uncertainty di alcune informazioni.
Informazioni imprecise non devono essere interpretate come una probabilità dell'informazione
Ad esempio, se io conosco l'inglese all'80% non vuole dire che ho una probabilità di conoscere l'inglese dell'80% ma vuol dire che conosco in parte (grossa parte) l'inglese che è diverso dalla probabilità di conoscerlo

Learning Classifier System (HOLLAND)

This is important because in chess games, for example, the mapping of all states and action is unfeasible.
function that providing some position tell us what to do
We have a set of rules that can be seen as a policy.
Input interface → create some message out of the real world.
Message List → Big DB of facts
Classifier List → Set of rules, these rules are working with the ML, they permit to create others Messages. example: if COND and COND … THEN action. We can add that action to the Message List

JH's Basic Structure



How can we create Rules ?
Holland's idea was to use GA to evolve the classifier and create new rules. Too much expensive in time and computation
Discovery component → something that create new rules

Machine Learning System with close links to reinforcement learning and genetic algorithms. In LCS an agent learns to perform a certain task by interacting with a partially unknown environment from which the agent receives feedback in the form of *reward.*
The incoming reward is exploited to guide the evolution of the agent's behavior, which in LCS, is represented by a set of **rules**, the ***classifiers.***

- Temporal difference learning is used to estimate the goodness of a classifier in terms of future reward
- GA are used to favor the reproduction and recombination of better classifier

LCS Rules

- Rules should work collaboratively
- The set of rules should **cover all the important situations (**not only handle very specialized situations)
- Only rules responsible for a chosen decision should be **reward**/penalized
- Lazy and inactive rules should be removed but the **very important** rules must not be removed

Online Approach → After each interaction, the system updates its population of classifiers based on the feedback received from the environment (e.g., rewards or penalties).
Classifier population interact with the environment → very expensive

Offline Approach → In the offline approach, the LCS learns from a static dataset or batch of data without direct interaction with the environment during the learning process.

SET COVERING

```
PRINTING SETS:
0) [False False  True False False]
1) [False  True False  True False]
2) [False False False False False]
3) [ True  True False False False]
4) [False False False  True False]
5) [False False  True False False]
6) [ True False False False False]
7) [False False False False False]
8) [False False False False False]
9) [False False False False False]
10) [False False False False False]
11) [False False False False  True]
------------------------
FUNC -->  -1
```

SIZE PROBLEM
Num points to cover: 5
Num sets: 12

For covering all the points, we have to find the sets which the union of true values cover all the 5 points.